

REST 練習 1：為什麼要 REST + 架構總覽

Lecture

REST (Representational State Transfer) 不是一個特定技術，而是一種設計 HTTP API 的風格 (architectural style)：把系統裡的每個東西都當作「資源」(Resource)，用一個 URL 代表它，然後用固定的一組 HTTP 動詞去操作它，而不是自己發明一堆自訂的操作名稱。

HTTP 動詞的標準語意 (這是整個 REST 課程的地基，之後每一題都會用到)：

動詞	語意	冪等性 (重複呼叫結果一樣嗎)	安全性 (會不會改動資料)
GET	查詢一個資源	是	是 (安全，不改資料)
POST	建立一個新資源	否 (重複呼叫可能建立出多筆)	否
PUT	整筆覆蓋更新一個資源	是 (重複送同一份資料，結果不變)	否
DELETE	刪除一個資源	是 (刪過一次後再刪，結果一樣是「不存在」)	否

「冪等」(Idempotent) 是 REST 設計的核心概念之一：呼叫端因為網路問題重送同一個請求時，冪等的動詞 (GET/PUT/DELETE) 不會因為重送而產生不良副作用，但 POST 的重送可能意外建立出重複資料 (rs06 會實際示範這個問題)。

SAP Classic REST 框架把「HTTP 協定細節」跟「你的業務邏輯」切成三層：

1. **SICF**：SAP Web Dispatcher/ICM 層級的路由設定，決定「這個 URL 路徑要交給哪個 ABAP Handler Class」——這一層沒有 ADT API，只能在 SAP GUI 手動掛載 (本課程唯一的例外)
2. **Application Class** (繼承 `CL_REST_HTTP_HANDLER`)：整個 Service 的「總機」，唯一要做的事是覆寫 `GET_ROOT_HANDLER`，決定要把 request 轉給哪個/哪些 Resource (rs02 開始會看到用 `CL_REST_ROUTER` 依路徑分流)
3. **Resource Class** (繼承 `CL_REST_RESOURCE`)：實際做事的地方，依 HTTP 動詞覆寫對應方法 (`GET/POST/PUT/DELETE`)，這裡才是寫業務邏輯的地方

這三層的分工讓你完全不用自己處理「怎麼解析 HTTP 請求」「例外要轉成什麼狀態碼」這些底層問題——`CL_REST_HTTP_HANDLER/CL_REST_RESOURCE` 已經幫你做好了，這也是為什麼這門課從第一題就可以直接寫業務邏輯，不用自己刻一個 HTTP Server。

學習目標

- 說得出 HTTP 動詞的語意 (GET 查詢 / POST 建立 / PUT 整筆更新 / DELETE 刪除) 與「冪等性」是什麼意思
- 理解 Classic REST 框架三個角色的分工：**SICF** (誰負責接這個 URL)、**Application Class** (`CL_REST_HTTP_HANDLER` 子類，決定誰處理這個 request)、**Resource Class** (`CL_REST_RESOURCE` 子類，實際處理某個 HTTP 動詞)

- 知道 `CL_REST_HTTP_HANDLER` 已經內建 CSRF 檢查、request/response 物件建立、例外轉 HTTP 狀態碼——子類別只需要覆寫 `GET_ROOT_HANDLER`
- 會用 ADT/SE24 建立繼承 `CL_REST_RESOURCE` 的類別，覆寫 `IF_REST_RESOURCE~GET`，用 `MO_RESPONSE->CREATE_ENTITY( )->SET_STRING_DATA( ... )` 回應純文字
- 完成一次「SICF 手動掛載 Handler Class」的完整流程——這是本課程唯一沒有 ADT API、必須在 SAP GUI 操作的步驟

為什麼不是「dump 內容就好」

跟 op11 學過的 `cl_salv_table` 不一樣，REST Service 沒有畫面、沒有使用者互動——呼叫方是另一支程式（前端 JS、Postman、第三方系統），溝通媒介是 HTTP request/response。傳統 ABAP 報表（FORM/Class）解決的是「使用者坐在 SAP GUI 前操作」的問題；REST 解決的是「系統跟系統對話」的問題。三個角色各自負責什麼：

角色	對應物件	負責什麼
SICF Service	<code>/sap/bc/zrest_training/rs01</code> （GUI 手動建立）	決定這個 URL 路徑由哪個 Handler Class 接手，相當於「總機」
Application Class	<code>ZCL_RS01_APP</code> 繼承 <code>CL_REST_HTTP_HANDLER</code>	收到 request 後，決定要交給哪個 Resource 處理（ <code>GET_ROOT_HANDLER</code> ）
Resource Class	<code>ZCL_RS01_HELLO</code> 繼承 <code>CL_REST_RESOURCE</code>	真正做事的地方，依 HTTP 動詞覆寫 <code>GET/POST/PUT/DELETE</code>

事前準備

- 確認你有 SAP GUI 的 SICF（Maintain Services）交易碼權限——這題最後要自己動手掛一次
- ADT 端物件已由課程準備好：`ZCL_RS01_APP`、`ZCL_RS01_HELLO`（`$TMP`），可以直接讀原始碼對照下面的說明

題目需求（對照已建好的答案物件）

1. `ZCL_RS01_APP`：INHERITING FROM `cl_rest_http_handler`，只覆寫 `IF_REST_APPLICATION~GET_ROOT_HANDLER`，回傳 `NEW zcl_rs01_hello( )`——目前只有一個資源，還不需要路由（下一題 `rs02` 才會用到 `CL_REST_ROUTER`）
2. `ZCL_RS01_HELLO`：INHERITING FROM `cl_rest_resource`，只覆寫 `IF_REST_RESOURCE~GET`：
 - 用 `MO_RESPONSE->CREATE_ENTITY( )` 拿到 `IF_REST_ENTITY`
 - `SET_STRING_DATA( )` 塞入純文字問候語（含目前伺服器時間）
 - `SET_CONTENT_TYPE( if_rest_media_type=>gc_text_plain )` 告訴瀏覽器這是純文字
 - 沒覆寫的 `POST/PUT/DELETE` 沿用父類別 `CL_REST_RESOURCE` 的預設實作，呼叫會直接回 `405 Method Not Allowed`——不用自己判斷「這個方法不支援」

SICF 手動掛載步驟（本課程僅此一題完整教學，之後每題比照辦理）

課程統一掛在 `/sap/bc/zrest_training/` 這個分類節點底下，每題各自一個子節點，避免 9 題下來把 `/sap/bc` 塞滿平行節點：

1. SAP GUI 執行交易碼 **SICF**

2. Hierarchy Type 選 **Service** · Service Path 留空後按執行 (顯示整棵樹)
3. 展開到 **default_host** → **sap** → **bc** · 在 **bc** 上按右鍵 → **New Sub-Element**
4. Service Name 填 **zrest_training** · Description 填「REST 課程」· 確定——這個節點是純目錄· 不用去 **Handler List** 掛任何類別· 也不用 **Activate**
5. 在 **zrest_training** node 上按右鍵 → **New Sub-Element** · Service Name 填 **rs01** · Description 填「REST 練習 1」· 確定
6. 開啟 **rs01** node · 切到 **Handler List** 頁籤 · 新增一筆 Handler Class : **ZCL_RS01_APP** · 儲存
7. 若圖示是灰色 (未啟用) · 在 **rs01** node 上按右鍵 → **Activate Service**
8. 測試：在 **rs01** node 上按右鍵 → **Test Service** (會開瀏覽器) · 或直接連 **http://<主機>:<port>/sap/bc/zrest_training/rs01/hello?sap-client=130**
9. 瀏覽器可能會跳出 Basic Auth 帳密框 · 輸入你的 SAP User/Password
10. 預期看到純文字回應 · 類似：**Hello REST! 現在伺服器時間是 14:32:07**

預期輸出 (範例)

```
Hello REST! 現在伺服器時間是 14:32:07
```

團隊實務備註

- **CL_REST_HTTP_HANDLER** 的 **GET_ROOT_HANDLER** 是**唯一**要覆寫的方法；
IF_HTTP_EXTENSION~HANDLE_REQUEST 父類已經寫死 · 不要嘗試覆寫它
- **IF_REST_RESOURCE~GET** 沒有 **IMPORTING/RETURNING** 參數 · 要透過繼承來的 **MO_REQUEST** (**IF_REST_REQUEST**) / **MO_RESPONSE** (**IF_REST_RESPONSE**) 保護屬性存取這次的 **request/response** · 跟一般方法「用參數傳資料」的習慣不同
- SICF Service 一旦掛好 · Handler Class 名稱不變 · 之後改 **ZCL_RS01_APP/ZCL_RS01_HELLO** 的程式碼**不用**重新掛載 · 改完啟用就生效——SICF 只是「指到哪個類別」 · 不是每次都要重掛
- **zrest_training** 這個父節點只需要建一次 · 之後 **rs02~rs09** 都只要在它底下新增子節點即可 · 不用重建

思考題

1. 如果把 **ZCL_RS01_HELLO** 的 **SET_CONTENT_TYPE** 那行刪掉會怎樣？ (提示：瀏覽器/Postman 判斷怎麼顯示回應內容 · 靠的是哪個 HTTP Header？)
2. 承接 op09 例外處理：如果 **IF_REST_RESOURCE~GET** 裡面發生未捕捉的例外 (例如 **SELECT SINGLE** 找不到資料卻硬要存取) · 呼叫端會收到什麼？ (提示：回頭看 **CL_REST_RESOURCE~IF_REST_HANDLER~HANDLE** 的 **TRY...CATCH cx_rest_exception**)
3. 為什麼 Resource Class 不用自己判斷「這個 URL 用了不支援的 HTTP 動詞」 · 而是直接繼承就有 405？

答案

見 **zcl_rs01_app.clas.abap** · **zcl_rs01_hello.clas.abap** (SAP 端物件 **ZCL_RS01_APP / ZCL_RS01_HELLO**) · SICF Service 路徑 **/sap/bc/zrest_training/rs01** · 需在各自的 SAP 系統手動掛載 (無法用快照複製 · 見上方「SICF 手動掛載步驟」)。